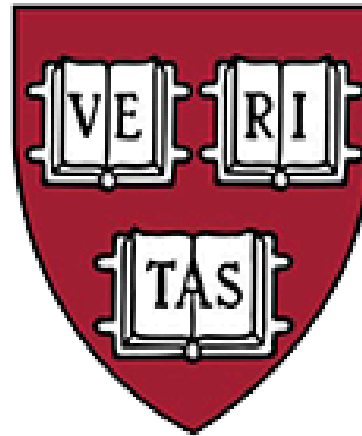




This is CS50.

7-SQL

Week 7 : SQL

Outline :

- Welcome!
- Flat-File Database
- Relational Databases
- SQL
- Indexes
- Using SQL in Python

• الهدف من ال week اننا نتعرف علي ال database وازاي بنقدر نتعامل معاها

Database

A Collection of related data organized for
creating , reading, updating and deleting

Flat - File Database

- As you have likely seen before, data can often be described in patterns of columns and rows.
- Spreadsheets like those created in Microsoft Excel and Google Sheets can be outputted to a csv or comma-separated values file.
- If you look at a csv file, you'll notice that the file is flat in that all of our data is stored in a single table represented by a text file. We call this form of data a flat-file database.
- Python comes with native support for csv files.

- من طرق تنظيم الداتا **spreadsheets / flat file / relational database**

في حالة ال spreadsheets ممكن اعدل فيهم من غير اي كود ولكن من عيوبها ان الادوات فيها محدوده عشان كده بنضطر نخطها علي **flat file**

- زي ما شفنا قبل كده ان البيانات غالبا بتبقا في نمط اعمدة و صفوف و كمثال في

ملفات ال CSV و هنلاحظ في النوع ده من الملفات ان البيانات بتاخذ شكل واحد ثابت و

بتتخزن كلها في جدول واحد و النوع ده من البيانات بيتسمى flat-file database

- لغة python بتدعم التعامل مع ملفات CSV (comma separated values)

What is Python

Python is a high-level, easy-to-read programming language used for:

- Web development
- Data analysis
- Artificial intelligence
- Automation
- Teaching programming

Variables in Python

You can create a variable just by assigning a value to a name:

```
name = "Hana"
```

```
age = 20
```

You don't need to write the type (like int, string).

You can print it like this:

```
print(f"My name is {name} and I'm {age} years old.")
```


If Statements

Python uses if, elif, and else to make decisions.

```
age = 18
```

```
if age >= 18:
```

```
    print("You are an adult.")
```

```
else:
```

```
    print("You are a minor.")
```

✓ Notice the : at the end of the condition, and the indentation under each block.

For Loops

for is used to loop through items like a list or range.

```
for i in range(5):  
    print(i)
```

Output:

0
1
2
3
4

Example with a list:

```
fruits = ["apple", "banana", "mango"]  
for fruit in fruits:  
    print(fruit)
```

- Download favorites.csv
- Upload it to your file explorer inside cs50.dev.

• لما انزله as csv file هيكون بين كل عمود والتاني الي بي فصلهم هو ال comma

```
Timestamp, language,problem
10/30/2023 13:38:01,Python,"Hello, World"
10/30/2023 13:38:03,Python,DNA
10/30/2023 13:38:03,Python,"Hello, World"
10/30/2023 13:38:05,Scratch,Scratch
10/30/2023 13:38:08,Python,Speller
10/30/2023 13:38:09,Python,"Hello, World"
10/30/2023 13:38:10,Python,Cash
10/30/2023 13:38:10,Python,Readability
10/30/2023 13:38:10,Python,Cash
10/30/2023 13:38:10,Python,Speller
10/30/2023 13:38:11,Python,Readability
10/30/2023 13:38:11,Python,Scratch
10/30/2023 13:38:12,Python,Filter
10/30/2023 13:38:12,Python,Tideman
10/30/2023 13:38:13,Scratch,Scratch
10/30/2023 13:38:13,Python,Caesar
```

• شكله في excel sheet

A	B	C	D
Timestamp	language	problem	
#####	Python	Hello, World	
#####	Python	DNA	
#####	Python	Hello, World	
#####	Scratch	Scratch	
#####	Python	Speller	
#####	Python	Hello, World	
#####	Python	Cash	
#####	Python	Readability	
#####	Python	Cash	
#####	Python	Speller	
#####	Python	Readability	
#####	Python	Scratch	
#####	Python	Filter	
#####	Python	Tideman	
#####	Scratch	Scratch	
#####	Python	Caesar	
#####	Python	Tideman	
#####	Python	Hello, World	
#####	Scratch	Mario	

column



row



A	B	C	D
Timestamp	language	problem	
#####	Python	Hello, World	
#####	Python	DNA	
#####	Python	Hello, World	
#####	Scratch	Scratch	
#####	Python	Speller	
#####	Python	Hello, World	
#####	Python	Cash	
#####	Python	Readability	
#####	Python	Cash	
#####	Python	Speller	
#####	Python	Readability	
#####	Python	Scratch	
#####	Python	Filter	
#####	Python	Tideman	
#####	Scratch	Scratch	
#####	Python	Caesar	
#####	Python	Tideman	
#####	Python	Hello, World	
#####	Scratch	Mario	

```
# Stores favorite in a variable
```

عشان اقدر اتحكم في الملف

```
import csv
```

```
# Open CSV file
```

```
with open("favorites.csv", "r") as file:
```

```
# Create reader
```

```
reader = csv.reader(file)
```

```
# Skip header row
```

```
next(reader)
```

```
# Iterate over CSV file, printing each favorite
```

```
for row in reader:
```

```
    favorite = row[1]
```

```
    print(favorite)
```



عملت open للفايل في ال "r" read mood لأن انا محتاج اقرأ منه

عملنا متغير اسمه reader وجوا ال csv فيه حاجه اسمها reader بتقرأ اي بيانات في الملف وتخزنها في المتغير علي شكل list لكل row

بعمل skip لأول صف عشان ده رأس الجدول وانا مش محتاجه الي هما (problem , language , Timestamp)

هنا هو بيطلع كل اللغات فقط (العمود الثاني)

output

- هيطبع كل اللغات الي موجوده عندي

```
$ python favorites.py
Python
Python
Python
Scratch
Python
Python
Python
Python
Python
Python
Python
Python
Python
Python
Scratch
Python
Python
Python
Scratch
Python
Scratch
Scratch
Python
C
```

- Python also allows you to index by the keys of a list. Modify your code as follows:

• فيه مشكله بتقابلني وهي ان مش دايمًا ال index لل favorite language بيكون 1 , ويمكن انسي هو انهو index من الاول

```
# Prints all favorites in CSV using csv.DictReader

import csv

# Open CSV file
with open("favorites.csv", "r") as file:

    # Create DictReader
    reader = csv.DictReader(file)

    # Iterate over CSV file, printing each favorite
    for row in reader:
        favorite = row["language"]
        print(favorite)
```

• عشان نحل المشكله دي استخدمنا DictReader
مش بترجع الصفوف ك list بترجعهم ك Dict فبيكون
عندي key و value ولو نلاحظ احنا كمان محتاجناش
نستخدم ال next

- To count the number of favorite languages expressed in the csv file, we can do the following:

عملنا هنا متغيرات قيمتهم ب 0 عشان نستخدمهم كل ما يقابلني في ال language
scratch هزود المتغير بتاعها ب 1
وهكذا مع c , python

• ولكن الطريقة دي مش احسن حاجه لأن انا ممكن يكون عندي لغات اكثر من كده

```
# Counts favorites using variables

import csv

# Open CSV file
with open("favorites.csv", "r") as file:

    # Create DictReader
    reader = csv.DictReader(file)

    # Counts
    scratch, c, python = 0, 0, 0

    # Iterate over CSV file, counting favorites
    for row in reader:
        favorite = row["language"]
        if favorite == "Scratch":
            scratch += 1
        elif favorite == "C":
            c += 1
        elif favorite == "Python":
            python += 1

    # Print counts
    print(f"Scratch: {scratch}")
    print(f"C: {c}")
    print(f"Python: {python}")
```


- Python allows us to use a dictionary to count the counts of each language. Consider the following improvement upon our code:

output

```
$ python favorites.py
Python: 280
Scratch: 40
C: 78
$
```

عملت هنا متغير بيساوي dict فاضيه ←

كل ما يقابل قيمه مختلفه هيعملها key ويبدأ يزود
في ال value بتاعتها ←

- Notice that the value in counts with the key favorite is incremented when it exists already. If it does not exist, we define counts[favorite] and set it to 1.

```
# Counts favorites using dictionary

import csv

# Open CSV file
with open("favorites.csv", "r") as file:

    # Create DictReader
    reader = csv.DictReader(file)

    # Counts
    counts = {}

    # Iterate over CSV file, counting favorites
    for row in reader:
        favorite = row["language"]
        if favorite in counts:
            counts[favorite] += 1
        else:
            counts[favorite] = 1

# Print counts
for favorite in counts:
    print(f"{favorite}: {counts[favorite]}")
```

- Python also allows sorting counts. Improve your code as follows:

output

```
$ python favorites.py
C: 78
Python: 280
Scratch: 40
$
```

لما كتبنا sorted هنا رتب هوليا بشكل ابجدي from A To Z يعني ترتيب علي حسب ال key مش ال values , لو حايبين نعكس الترتيب ويكون from Z To A نكتب (counts , reverse=True)

reverse

```
$ python favorites.py
Scratch: 40
Python: 280
C: 78
$
```

```
# Sorts favorites by key
```

```
import csv
```

```
# Open CSV file
```

```
with open("favorites.csv", "r") as file:
```

```
# Create DictReader
```

```
reader = csv.DictReader(file)
```

```
# Counts
```

```
counts = {}
```

```
# Iterate over CSV file, counting favorites
```

```
for row in reader:
```

```
    favorite = row["language"]
```

```
    if favorite in counts:
```

```
        counts[favorite] += 1
```

```
    else:
```

```
        counts[favorite] = 1
```

```
# Print counts
```

```
for favorite in sorted(counts):
```

```
    print(f"{favorite}: {counts[favorite]}")
```

- If you look at the parameters for the sorted function in the Python documentation, you will find it has many built-in parameters. You can leverage some of these built-in parameters as follows:
- Notice the arguments passed to sorted. The key argument allows you to tell Python the method you wish to use to sort items. In this case counts.get is used to sort by the values. reverse=True tells sorted to sort from largest to smallest.

output

```
$ python favorites.py
Python: 280
C: 78
Scratch: 40
$ █
```

في الحالة دي احنا بنعمل sort علي اساس ال vlaues مش ال keys
 عن طريق ال counts.get بترجعلي ال values وبيتعملها sort
 و reverse=True عشان يبقي الترتيب من الكبير للصغير

```
# Sorts favorites by value using .get

import csv

# Open CSV file
with open("favorites.csv", "r") as file:

    # Create DictReader
    reader = csv.DictReader(file)

    # Counts
    counts = {}

    # Iterate over CSV file, counting favorites
    for row in reader:
        favorite = row["language"]
        if favorite in counts:
            counts[favorite] += 1
        else:
            counts[favorite] = 1

    # Print counts
    for favorite in sorted(counts, key=counts.get, reverse=True):
        print(f"{favorite}: {counts[favorite]}")
```

- Python has numerous libraries that we can utilize in our code. One of these libraries is collections, from which we can import Counter. Counter will allow you to access the counts of each language without the headaches of all the if statements seen in our previous code. You can implement as follows:

`import Counter`

• بتخلينا نستخدم مميزات و functions خاصة بال counting

عشان المتغير ياخذ المميزات ال Counter ←

وهنا استعملنا `most_common()` عشان نطبع اللغات من الاكثر تكرار

output

```
$ python favorites.py
Python: 280
C: 78
Scratch: 40
$ █
```

```
# Sorts favorites by value using .get
```

```
import csv
```

```
from collections import Counter
```

```
# Open CSV file
```

```
with open("favorites.csv", "r") as file:
```

```
# Create DictReader
```

```
reader = csv.DictReader(file)
```

```
# Counts
```

```
counts = Counter()
```

```
# Iterate over CSV file, counting favorites
```

```
for row in reader:
```

```
    favorite = row["language"]
```

```
    counts[favorite] += 1
```

```
# Print counts
```

```
for favorite, count in counts.most_common():
```

```
    print(f"{favorite}: {count}")
```

- We can also get the count of the popularity of a specific problem in the course:

• وده لو احنا حبين نجيب عدد الناس الي فضلت problem معينه

output

```
$ python favorites.py
Favorite: Cash
Cash: 24
$ □
```

- هناخد input بأسم ال problem مثلا "cash" ونشوف من الداتا عندنا هي ا تكررت كام مره

```
# Gets a specific count

import csv

from collections import Counter

# Open CSV file
with open("favorites.csv", "r") as file:

    # Create DictReader
    reader = csv.DictReader(file)

    # Counts
    counts = Counter()

    # Iterate over CSV file, counting favorites
    for row in reader:
        favorite = row["problem"]
        counts[favorite] += 1

# Print count
favorite = input("Favorite: ")
print(f"{favorite}: {counts[favorite]}")
```